CHAPTER 1 : INTRODUCTION

Definition: A computer program is said to **learn** from **experience E** with respect to some **class of tasks T** and **performance measure P**, if its performance at tasks in T, as measured by P, improves with experience E.

For example, a computer program that learns to play checkers might improve its performance as measured by its ability to win at the class of tasks involving playing checkers games, through experience obtained by playing games against itself. In general, to have a well-defined learning problem, we must identity these three features:

1. the class of tasks,
2. the measure of performance to be improved, and
3. the source of experience.

A checkers learning problem:

- **Task T:** playing checkers
- **Performance measure P:** percent of games won against opponents
- **Training experience E:** playing practice games against itself

We can specify many learning problems in this fashion, such as learning to recognize handwritten words, or learning to drive a robotic automobile autonomously.

A handwriting recognition learning problem:

Task T: recognizing and classifying handwritten words within images

Performance measure P: percent of words correctly classified

Training experience E: a database of handwritten words with given classifications

A robot driving learning problem:

Task T: driving on public four-lane highways using vision sensors

Performance measure P: average distance travelled before an error (as judged by human overseer)

DEEPAK RAJPUT

Training experience E: a sequence of images and steering commands recorded while observing a human driver

CHAPTER 2: CONCEPT LEARNING AND THE GENERAL-TO-SPECIFIC ORDERING

INTRODUCTION

Concept learning. Inferring a Boolean valued function from training examples of its input and output.

A CONCEPT LEARNING TASK

The attribute EnjoySport indicates whether or not Aldo enjoys his favourite water sport on this day. The task is to learn to predict the value of EnjoySport for an arbitrary day, based on the values of its other attributes.

Let us begin by considering a simple representation in which each hypothesis consists of a conjunction of constraints on the instance attributes. In particular, let each hypothesis be a vector of six constraints, specifying the values of the six attributes Sky, AirTemp, Humidity, Wind, Water, and Forecast. For each attribute, the hypothesis will either 0 indicate by a "?' that any value is acceptable for this attribute, 0 specify a single required value (e.g., Warm) for the attribute, or 0 indicate by a "0" that no value is acceptable.

- indicate by a "?' that any value is acceptable for this attribute,
- specify a single required value (e.g., Warm) for the attribute, or
- indicate by a "Φ" that no value is acceptable.

If some instance x satisfies all the constraints of hypothesis h, then h classifies x as a positive example (h(x) = 1). To illustrate, the hypothesis that Aldo enjoys his favourite sport only on cold days with high humidity (independent of the values of the other attributes) is represented by the expression

(?, Cold, High, ?, ?, ?)

Example Sky AirTemp Humidity Wind Water Forecast EnjoySport

1 Sunny Warm Normal Strong Warm Same Yes

2 Sunny Warm High Strong Warm Same Yes

3 Rainy Cold High Strong Warm Change No

DEEPAK RAJPUT

4 Sunny Warm High Strong Cool Change Yes

The most general hypothesis-that every day is a positive example-is represented by

(?, ?, ?, ?, ?, ?)

and the most specific possible hypothesis-that no day is a positive example-is represented by

(Φ, Φ, Φ, Φ, Φ, Φ)

Notation

The set of items over which the concept is defined is called the set of instances, which we denote by X. In the current example, X is the set of all possible days, each represented by the attributes Sky, AirTemp, Humidity, Wind, Water, and Forecast. The concept or function to be learned is called the target concept, which we denote by c. In general, c can be any boolean-valued function defined over the instances X; that is, c : X -> {O, 1}.

In the current example, the target concept corresponds to the value of the attribute EnjoySport

(i.e., c(x) = 1 if EnjoySport = Yes, and c(x) = 0 if EnjoySport = No).

- Given: 0
  - Instances X: Possible days, each described by the attributes
    - Sky (with possible values Sunny, Cloudy, and Rainy),
    - AirTemp (with values Warm and Cold),
    - Humidity (with values Normal and High),
    - Wind (with values Strong and Weak),
    - Water (with values Warm and Cool), and
    - Forecast (with values Same and Change).
  - Hypotheses H: Each hypothesis is described by a conjunction of constraints on the attributes Sky, AirTemp, Humidity, Wind, Water, and Forecast. The constraints may be "?" (any value is acceptable), " Φ”(no value is acceptable), or a specific value.

DEEPAK RAJPUT

- Target concept c: EnjoySport : X -> {0,1}
- Training examples D: Positive and negative examples of the target function (see Table 2.1).
- Determine:
    - hypothesis h in H such that h(x) = c(x) for all x in X.

Instances for which c(x) = 1 are called positive examples, or members of the target concept. Instances for which c(x) = 0 are called negative examples, or nonmembers of the target concept.

In general, each hypothesis h in H represents a boolean-valued function defined over X; that is, h : X -> {O, 1}. The goal of the learner is to find a hypothesis h such that h(x) = c(x) for all x in X.

The Inductive Learning Hypothesis

The inductive learning hypothesis. Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over other unobserved examples.

General-to-Specific Ordering of Hypotheses(Not Necessary)

To illustrate the general-to-specific ordering, consider the two hypotheses

h1 = (Sunny, ?, ?, Strong, ?, ?)

h2 = (Sunny, ?, ?, ?, ?, ?)

Now consider the sets of instances that are classified positive by h1 and by h2. Because h2 imposes fewer constraints on the instance, it classifies more instances as positive. In fact, any instance classified positive by h1 will also be classified positive by h2. Therefore, we say that h2 is more general than h1.

This intuitive "more general than" relationship between hypotheses can be defined more precisely as follows. First, for any instance x in X and hypothesis h in H, we say that x satisfies h if and only if h(x) = 1. We now define the more_general_than_or_equal_to relation in terms of the sets of instances that satisfy the two hypotheses: Given hypotheses h_j and h_k, h_j is more_general_than_or_equal_to h_k if and only if any instance that satisfies h_k also satisfies h_i.

Definition: Let h_j and h_k be boolean-valued functions defined over X. Then h_j is more_general_than_or_equal_to h_k (written $h_j >=_g h_k$) if and only if

DEEPAK RAJPUT

$$(\forall x \in X)[(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$$

We will also find it useful to consider cases where one hypothesis is strictly more general than the other. Therefore, we will say that h_j is (strictly) more_general_than_or_equal_to h_k (written $h_j >_g h_k$) if and only if ($h_j >=_g h_k$) ($h_k >!=_g h_j$). Finally, we will sometimes find the inverse useful and will say that hj is more_specific_than h_k when h_k is more_general_than h_j.

The $>=_g$ relation is important because it provides a useful structure over the hypothesis space H for any concept learning problem. The following sections present concept learning algorithms that take advantage of this partial order to efficiently organize the search for hypotheses that fit the training data.

FIND-S: FINDING A MAXIMALLY SPECIFIC HYPOTHESIS

Find-S Algorithm is a basic concept learning algorithm which find the most specific hypothesis that fits all the positive example.

The **find-S Algorithm** also starts with most specific hypothesis & generalizes this hypothesis each time. It fails classify and observe positive training data.

Hence **find-S Algorithm** moves from the most specific hypothesis to most generalizes hypothesis.

DEEPAK RAJPUT

CHAPTER 6: BAYESIAN LEARNING

INTRODUCTION

Bayesian learning methods are relevant to our study of machine learning for two different reasons. First, Bayesian learning algorithms that calculate explicit probabilities for hypotheses, such as the naive Bayes classifier, are among the most practical approaches to certain types of learning problems.

The second reason that Bayesian methods are important to our study of machine learning is that they provide a useful perspective for understanding many learning algorithms that do not explicitly manipulate probabilities.

Features of Bayesian learning methods include:

- Each observed training example can incrementally decrease or increase the estimated probability that a hypothesis is correct. This provides a more flexible approach to learning than algorithms that completely eliminate a hypothesis if it is found to be inconsistent with any single example.
- Prior knowledge can be combined with observed data to determine the final probability of a hypothesis. In Bayesian learning, prior knowledge is provided by asserting

  (1) a prior probability for each candidate hypothesis, and

  (2) a probability distribution over observed data for each possible hypothesis.
- Bayesian methods can accommodate hypotheses that make probabilistic predictions (e.g., hypotheses such as "this pneumonia patient has a 93% chance of complete recovery").
- New instances can be classified by combining the predictions of multiple hypotheses, weighted by their probabilities.
- Even in cases where Bayesian methods prove computationally intractable, they can provide a standard of optimal decision making against which other practical methods can be measured.

BAYES THEOREM

P(h) to denote the initial probability that hypothesis h holds, before we have observed the training data. P(h) is often called the prior probability of h.

P(D) to denote the prior probability that training data D will be observed (i.e., the probability of D given no knowledge about which hypothesis holds).

DEEPAK RAJPUT

P(D|h) to denote the probability of observing data D given some world in which hypothesis h holds.

P(xly) to denote the probability of x given y.

P(h|D) that h holds given the observed training data D. P (h 1 D) is called the posterior probability of h.

Bayes theorem is the cornerstone of Bayesian learning methods because it provides a way to calculate the posterior probability P(h|D), from the prior probability P(h), together with P(D) and P(D(h)).

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

(Formula used in example)

Example :- (10 marks confirm)

S. No.	Outlook	Temperature	Humidity	Windy	Play on
1	Rainy	Hot	High	FALSE	No
2	Rainy	Hot	High	TRUE	No
3	Overcast	Hot	High	FALSE	Yes
4	Sunny	Mild	High	FALSE	Yes
5	Sunny	Cool	Normal	FALSE	Yes
6	Sunny	Cool	Normal	TRUE	No
7	Overcast	Cool	Normal	TRUE	Yes
8	Rainy	Mild	High	FALSE	No
9	Rainy	Cool	Normal	FALSE	Yes
10	Sunny	Mild	Normal	FALSE	Yes
11	Rainy	Mild	Normal	TRUE	Yes
12	Overcast	Mild	High	TRUE	Yes
13	Overcast	Hot	Normal	FALSE	Yes
14	Sunny	Mild	High	TRUE	No

DEEPAK KAPOOT

Outlook

	Yes	No	Probability of Yes	Probability of No
Rainy	2	3		
Sunny	3	2		
Overcast	4	0		
Total	9	5	100	100

Temperature

	Yes	No	Probability of Yes	Probability of No
Hot	2	2		
Mild	3	1		
Cool	4	2		
Total	9	5	100	100

Humidity

	Yes	No	Probability of Yes	Probability of No
High	3	4		
Normal	6	1		
Total	9	5	100	100

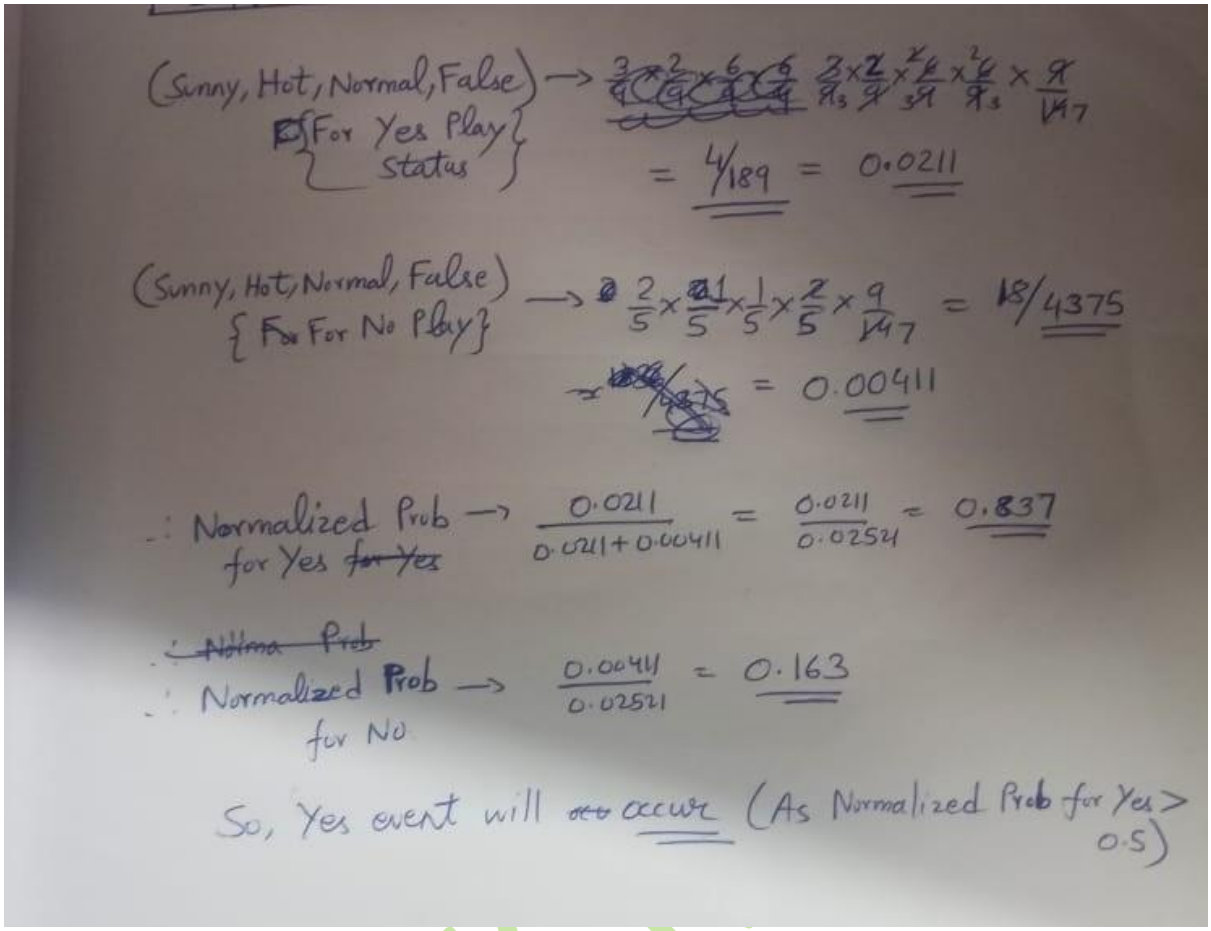
Windy

	Yes	No	Probability of Yes	Probability of No
TRUE	6	2		
FALSE	3	3		
Total	9	5	100	100

Play On

	Number	Probability
Yes	9	(9/14)
No	5	(5/14)
Total	14	

DEEPAK RAJPUT



Pattern Recognition and Machine Learning

The Curse of Dimensionality

Curse of Dimensionality describes the explosive nature of increasing data dimensions and its resulting exponential increase in computational efforts required for its processing and/or analysis.

As the dimensionality increases, the number of data points required for good performance of any machine learning algorithm increases exponentially.

Scatter plot of the oil flow data for input variables x6 and x7, in which red denotes the 'homogenous' class, green denotes the 'annular' class, and blue denotes the 'laminar' class. Our goal is to classify the new test point denoted by 'x'.

DEEPAK

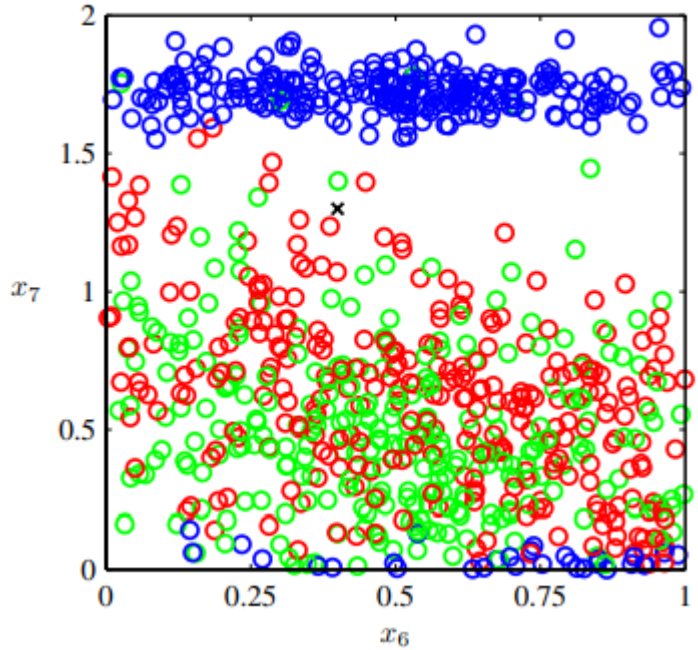
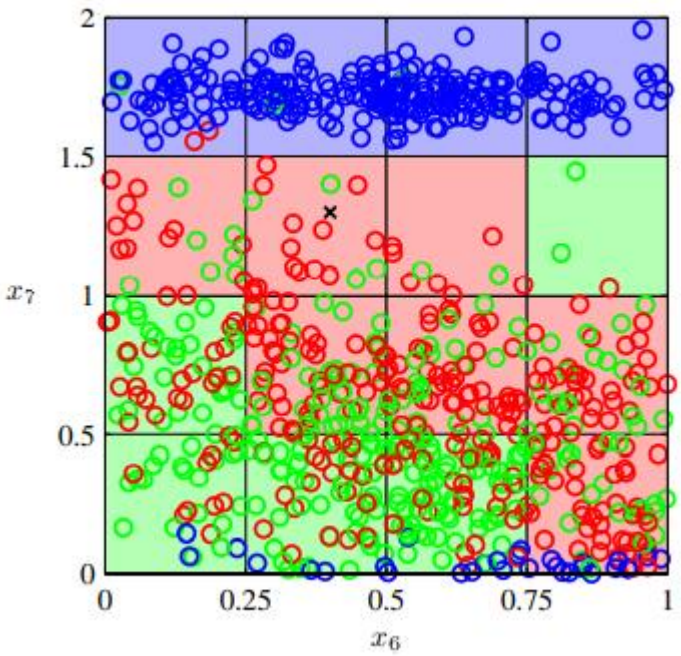


Illustration of a simple approach to the solution of a classification problem in which the input space is divided into cells and any new test point is assigned to the class that has a majority number of representatives in the same cell as the test point. As we shall see shortly, this simplistic approach has some severe shortcomings.



The problem with an exponentially large number of cells is that we would need an exponentially large quantity of training data in order to ensure that the cells are not empty. Clearly, we have no hope of applying such a technique in a space of more than a few variables, and so we need to find a more sophisticated approach

The Art and Science of Algorithms that Make Sense of Data

Kinds of feature

Consider two features, one describing a person's age and the other their house number. Both features map into the integers, but the way we use those features can be quite different. Calculating the average age of a group of people is meaningful, but an average house number is probably not very useful! In other words, what matters is not just the domain of a feature, but also the range of permissible operations. These, in turn, depend on whether the feature values are expressed on a meaningful **scale**. Despite appearances, house numbers are not really integers but **ordinals**: we can use them to determine that number 10's neighbours are number 8 and number 12, but we cannot assume that the distance between 8 and 10 is the same as the distance between 10 and 12. Because of the absence of a linear scale it is not meaningful to add or subtract house numbers, which precludes operations such as averaging.

Calculations on features

Three main categories are **statistics of central tendency, statistics of dispersion** and **shape statistics.** Each of these can be interpreted either as a theoretical property of an unknown population or a concrete property of a given sample – here we will concentrate on sample statistics.

Starting with statistics of central tendency, the most important ones are:

- the **mean** or average value;
- the **median**, which is the middle value if we order the instances from lowest to highest feature value; and
- the **mode**, which is the majority value or values.

Imagine a swimmer who swims the same distance d on two different days, taking a seconds one day and b seconds the next. On average, it took her therefore $c = (a + b)/2$ seconds, with an average speed of $d/c = 2d/(a + b)$. Notice how this average speed is not calculated as the normal or **arithmetic mean** of

DEEPAK RAJPUT

the speeds, which would yield (d/a + d/b)/2: to calculate average speed over a fixed distance we use a different mean called the **harmonic mean**. Given two numbers x and y (in our swimming example these are the speeds on either day, d/a and d/b), the harmonic mean h is defined as

Since 1/h(x, y) = (1/x + 1/y)/2, we observe that calculating the harmonic mean on a scale with unit u corresponds to calculating the arithmetic mean on the **reciprocal scale** with unit 1/u.

In the example, speed with fixed distance is expressed on a scale reciprocal to the time scale, and since we use the arithmetic mean to average time, we use the harmonic mean to average speed. (If we average speed over a fixed time interval this is expressed on the same scale as distance and thus we would use the arithmetic mean.)

A good example of where the harmonic mean is used in machine learning arises when we average precision and recall of a classifier. Remember that precision is the proportion of positive predictions that is correct (prec = TP/(TP + FP)), and recall is the proportion of positives that is correctly predicted (rec = TP/(TP + FN)). Suppose we first calculate the number of mistakes averaged over the classes: this is the arithmetic mean Fm = (FP + FN)/2. We can then derive

We recognise the last term as the harmonic mean of precision and recall. Since the enumerator of both precision and recall is fixed, taking the arithmetic mean of the denominators corresponds to taking the harmonic mean of the ratios. In information retrieval this harmonic mean of precision and recall is very often used and called the **F-measure**.

Yet other means exist for other scales. In music, going from one note to a note one octave higher corresponds to doubling the frequency. So frequencies f and 4f are two octaves apart, and it makes sense to take the octave in between with frequency 2f as their mean. This is achieved by the **geometric mean**, which is defined as *

Since * it follows that the geometric mean corresponds to the arithmetic mean on a logarithmic scale. All these means have in common that the mean of two values is an interediate value, and that they can easily be extended to more than two values.

The second kind of calculation on features are statistics of **dispersion** or '**spread'**. Two well-known statistics of dispersion are the **variance** or **average**

DEEPAK RAJPUT

squared deviation from the (arithmetic) mean, and its square root, the **standard deviation**.

Variance and **standard deviation** essentially measure the same thing, but the latter has the advantage that it is expressed on the same scale as the feature itself. **For example,** the variance of the body weight in kilograms of a group of people is measured in kg 2 (kilograms-squared), whereas the standard deviation is measured in kilograms. The absolute difference between the mean and the median is never larger than the standard deviation – this is a consequence of **Chebyshev's inequality**, which states that at most 1/k 2 of the values are more than k standard deviations away from the mean.

A simpler dispersion statistic is the difference between maximum and minimum value, which is called the **range**. A natural statistic of central tendency to be used with the range is the **midrange point**, which is the mean of the two extreme values. These definitions assume a linear scale but can be adapted to other scales using suitable transformations. **For example**, for a feature expressed on a logarithmic scale, such as frequency, we would take the ratio of the highest and lowest frequency as the range, and the harmonic mean of these two extremes as the midrange point.

Other statistics of dispersion include **percentiles**. The p-th percentile is the value such that p per cent of the instances fall below it. If we have 100 instances, the 80th percentile is the value of the 81st instance in a list of increasing values. If p is a multiple of 25 the percentiles are also called **quartiles**, and if it is a multiple of 10 the percentiles are also called **deciles**. Note that the 50th percentile, the 5th decile and the second quartile are all the same as the median. **Percentiles**, **deciles** and **quartiles** are special cases of **quantiles**. Once we have quantiles we can measure dispersion as the distance between different quantiles. For instance, the **interquartile range** is the difference between the third and first quartile (i.e., the 75th and 25th percentile).

Example 10.1 (Percentile plot). Suppose you are learning a model over an instance space of countries, and one of the features you are considering is the gross domestic product (GDP) per capita. Figure 10.1 shows a so-called **percentile plot** of this feature. In order to obtain the p-th percentile, you intersect the line y = p with the dotted curve and read off the corresponding percentile on the x-axis. Indicated in the figure are the 25th, 50th and 75th percentile. Also indicated is the mean (which has to be calculated from the raw data). As you can see, the mean is considerably higher than the median; this is

DEEPAK RAJPUT

mainly because of a few countries with very high GDP per capita. In other words, the mean is more sensitive to **outliers** than the median, which is why the median is often preferred to the mean for skewed distributions like this one.

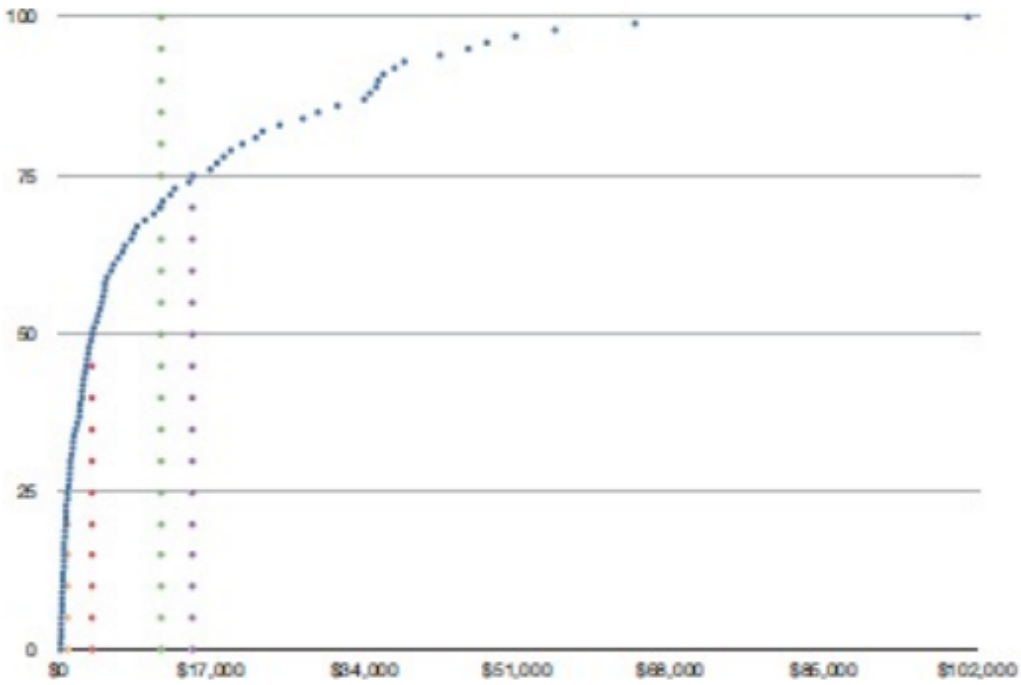


Figure 10.1. Percentile plot of GDP per capita for 231 countries (data obtained from www.wolframalpha.com by means of the query 'GDP per capita'). The vertical dotted lines indicate, from left to right: the **first quartile** ($900); the **median** ($3600); the **mean** ($11284); and the **third quartile** ($14,750). The **interquartile range** is $13 850, while the **standard deviation** is $16,189.

Categorical, ordinal and quantitative features

Features with an ordering but without scale are called **ordinal features.**

Features with a meaningful numerical scale are called **quantitative**.

Features without ordering or scale are called **categorical features** (or sometimes 'nominal' features). They do not allow any statistical summary except the mode. One subspecies of the categorical features is the **Boolean feature**, which maps into the truth values **true** and **false**.

Feature transformations

Feature transformations aim at improving the utility of a feature by removing, changing or adding information. We could order feature types by the amount of

detail they convey: quantitative features are more detailed than ordinal ones, followed by categorical features, and finally Boolean features.

Binarisation transforms a categorical feature into a set of Boolean features, one for each value of the categorical feature. This loses information since the values of a single categorical feature are mutually exclusive, but is sometimes needed if a model cannot handle more than two feature values.

Unordering trivially turns an ordinal feature into a categorical one by discarding the ordering of the feature values. This is often required since most learning models cannot handle ordinal features directly.

Thresholding transforms a quantitative or an ordinal feature into a Boolean feature by finding a feature value to split on.

Without knowing how this feature is to be used in a model, the most sensible thresholds are the statistics of central tendency such as the mean and the median. This is referred to as **unsupervised thresholding**.

Discretisation transforms a quantitative feature into an ordinal feature. Each ordinal value is referred to as a bin and corresponds to an interval of the original quantitative feature.

Unsupervised discretisation methods typically require one to decide the number of bins beforehand. A simple method that often works reasonably well is to choose the bins so that each bin has approximately the same number of instances: this is referred to as **equal-frequency discretisation**.

Another unsupervised discretisation method is **equal-width discretisation**, which chooses the bin boundaries so that each interval has the same width.

Switching now to **supervised discretisation** methods, we can distinguish between **top–down** or **divisive** discretisation methods on the one hand, and **bottom–up** or **agglomerative** discretisation methods on the other. Divisive methods work by progressively splitting bins, whereas agglomerative methods proceed by initially assigning each instance to its own bin and successively merging bins. In either case an important role is played by the **stopping criterion**, which decides whether a further split or merge is worthwhile.

DEEPAK RAJPUT

Feature normalisation is often required to neutralise the effect of different quantitative features being measured on different scales. If the features are approximately normally distributed, we can convert them into **z-scores** by centring on the mean and dividing by the standard deviation.

Feature calibration is understood as a supervised feature transformation adding a meaningful scale carrying class information to arbitrary features. This has a number of important advantages. For instance, it allows models that require scale, such as linear classifiers, to handle categorical and ordinal features.

Feature construction and selection

Forward selection methods start with an empty set of features and add features to the set one at a time, as long as they improve the performance of the model. **Backward elimination** starts with the full set of features and aims at improving performance by removing features one at a time.

One of the best-known algebraic feature construction methods is **principal component analysis (PCA)**. Principal components are new features constructed as linear combinations of the given features. The first principal component is given by the direction of maximum variance in the data; the second principal component is the direction of maximum variance orthogonal to the first component, and so on.

PCA can be explained in a number of different ways: here, we will derive it by means of the **singular value decomposition (SVD)**. Any n-by-d matrix can be uniquely written as a product of three matrices with special properties:

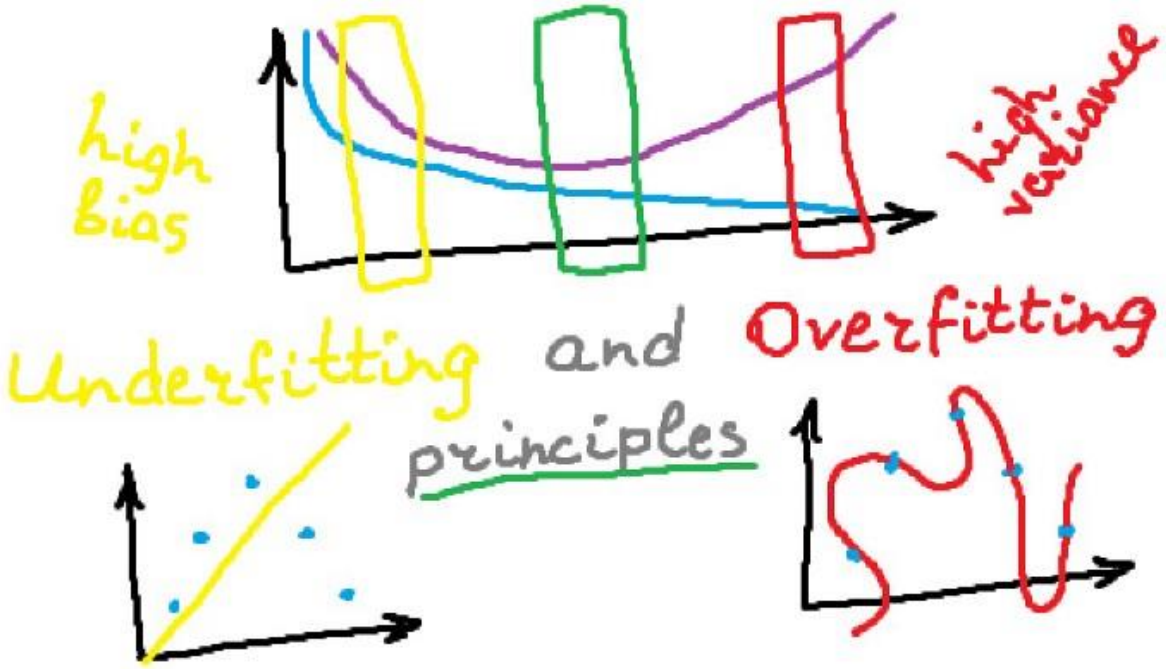
$$\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^{\mathrm{T}}$$

Here, U is an n-by-r matrix, Σ is an r-by-r matrix and V is an d-by-r matrix (for the moment we will assume r = d < n). Furthermore, U and V are orthogonal (hence rotations) and Σ is diagonal (hence a scaling). The columns of U and V are known as the left and right singular vectors, respectively; and the values on the diagonal of Σ are the corresponding singular values. It is customary to order the columns of V and U so that the singular values are decreasing from top-left to bottom-right.

DEEPAK RAJPUT

Overfitting and Underfitting Principles

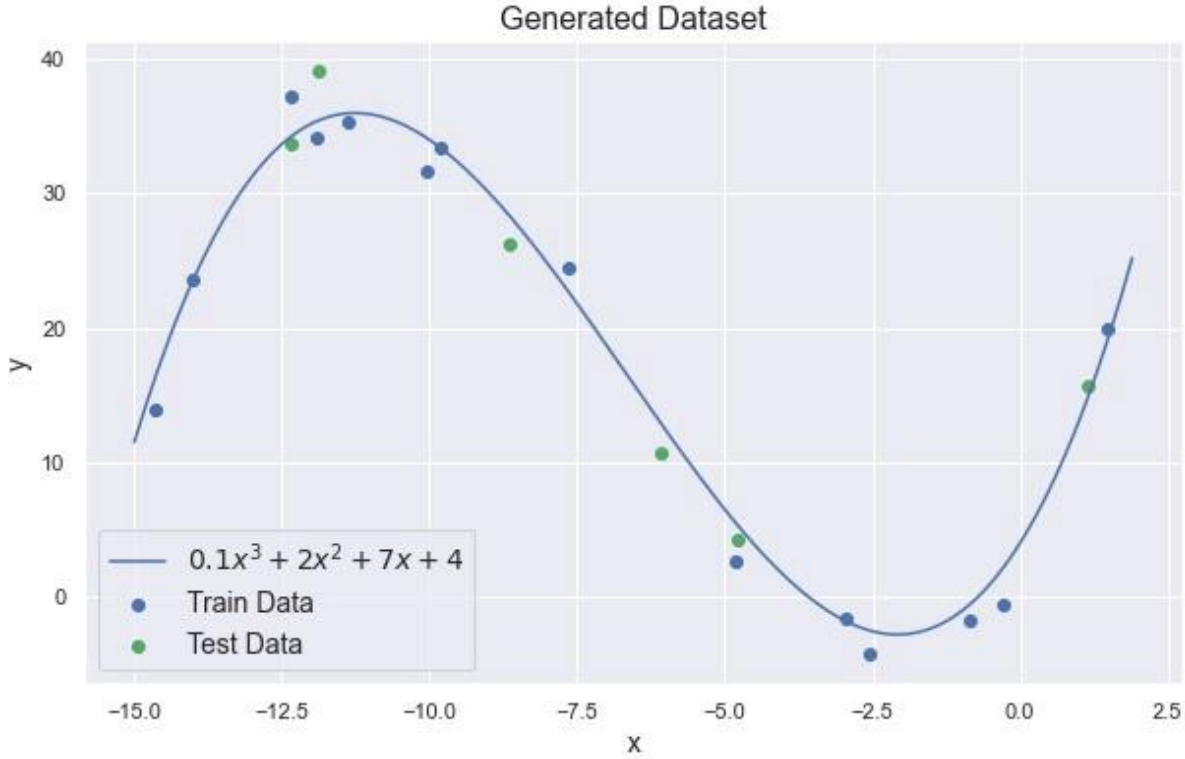
Understand basic principles of underfitting and overfitting and why you should use particular techniques to **deal with them**



Underfitting and overfitting principles. Image by Author

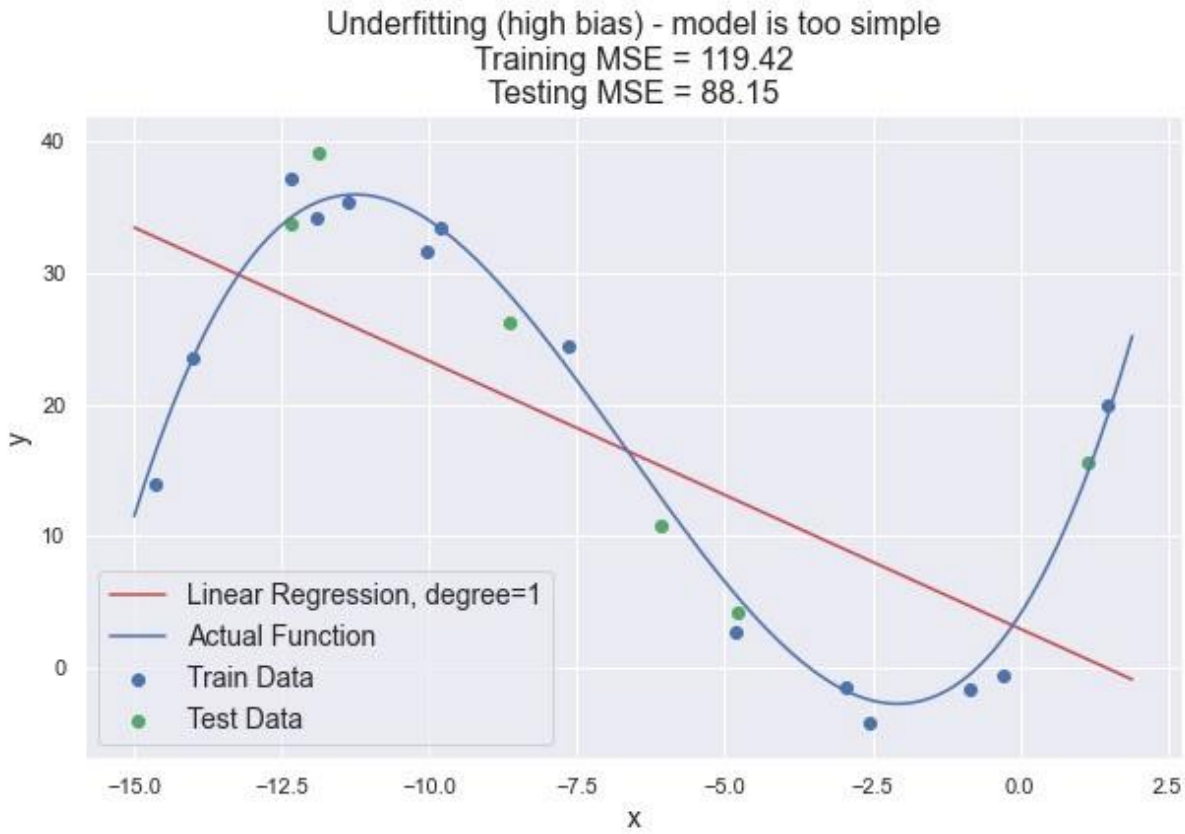
DEEPAK

Underfitting and Overfitting and Bias/Variance Trade-off



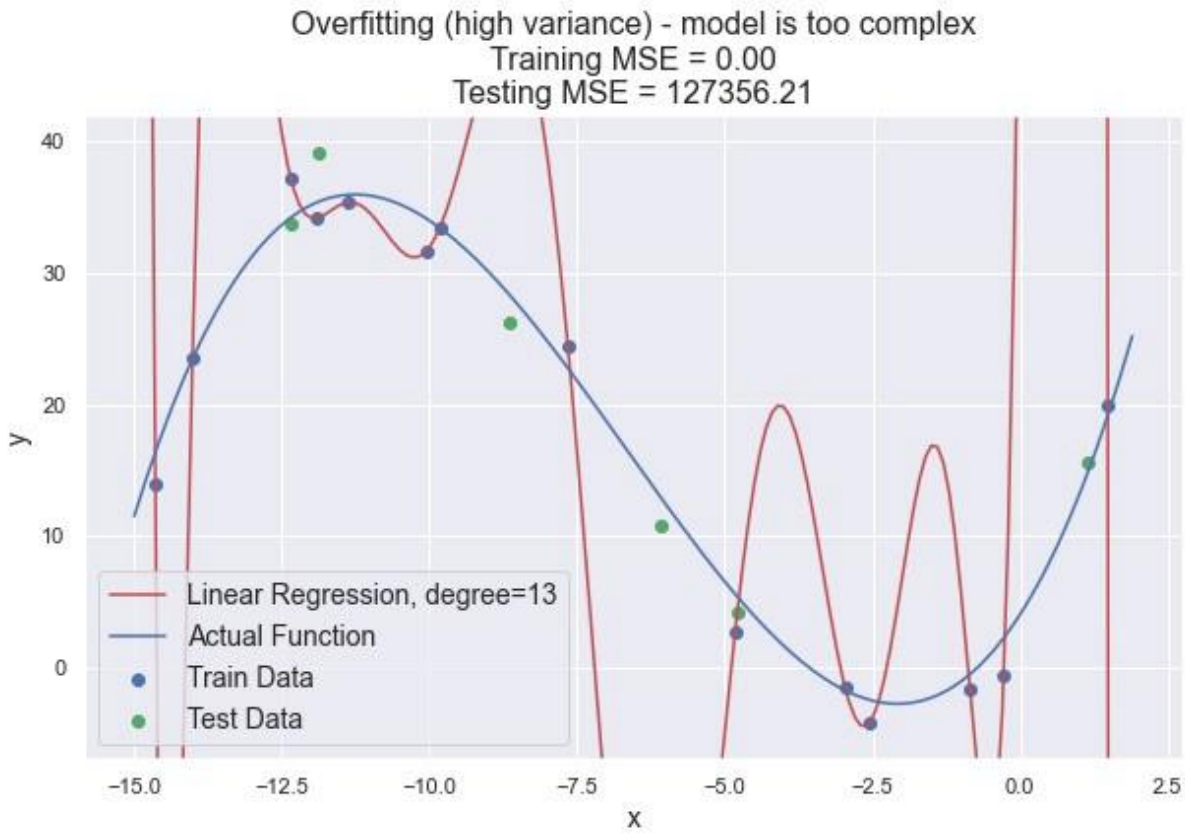
Generated dataset. Image by Author

Underfitting is a situation when your model is **too simple** for your data. More formally, your hypothesis about data distribution is wrong and too simple — for example, your data is quadratic and your model is linear. This situation is also called **high bias**. This means that your algorithm can do accurate predictions, but the initial assumption about the data is incorrect.



Underfitting. The linear model trained on cubic data. Image by Author

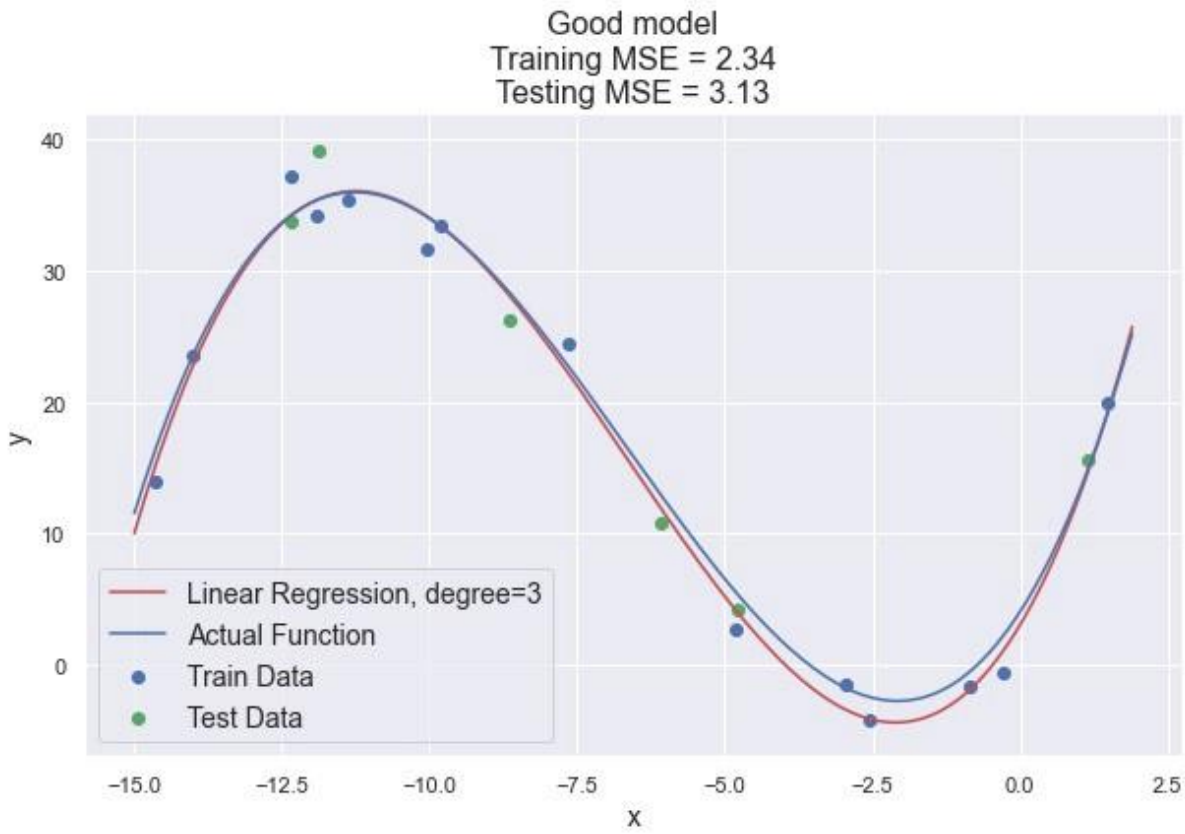
Opposite, **overfitting** is a situation when your model is **too complex** for your data. More formally, your hypothesis about data distribution is wrong and too complex — for example, your data is linear and your model is high-degree polynomial. This situation is also called **high variance**. This means that your algorithm can't do accurate predictions — changing the input data only a little, the model output changes very much.



Overfitting. The 13-degree polynomial model trained on cubic data. Image by Author

These are two extremes of the same problem and the optimal solution always lies somewhere in the middle.

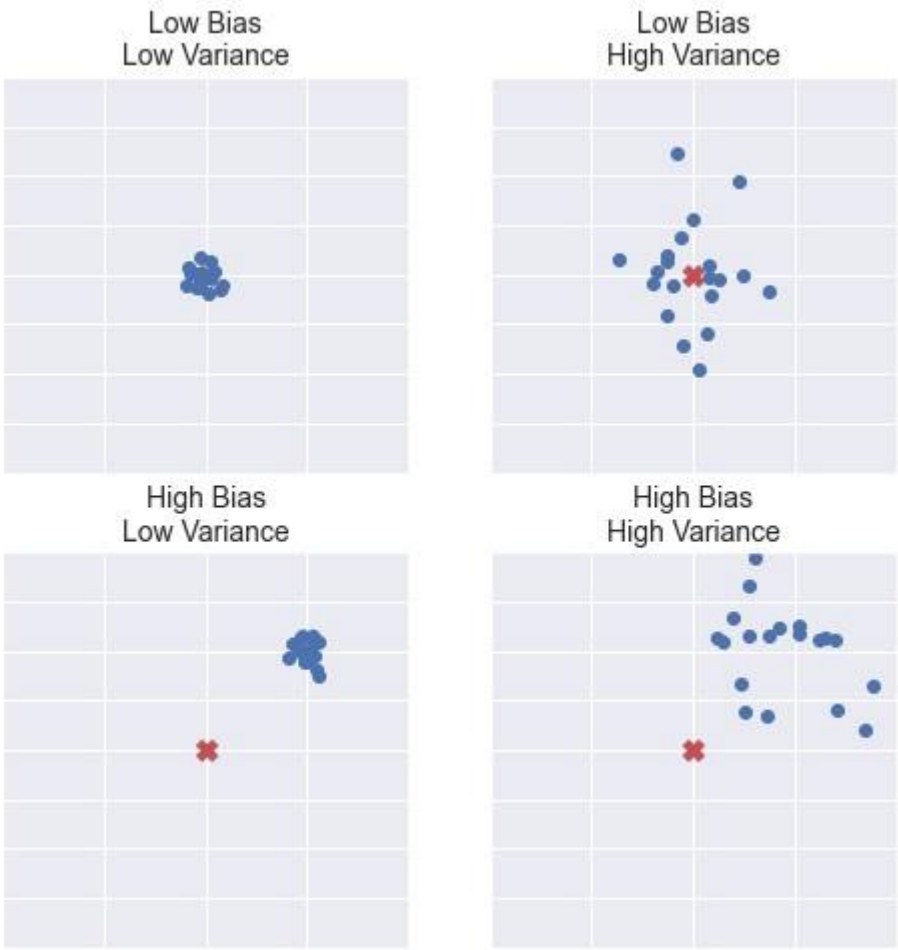
DEEPAK K



Good model. The cubic model trained on cubic data. Image by Author

I will not talk much about bias/variance trade-off (you can read the basics in this article), but let me briefly mention possible options:

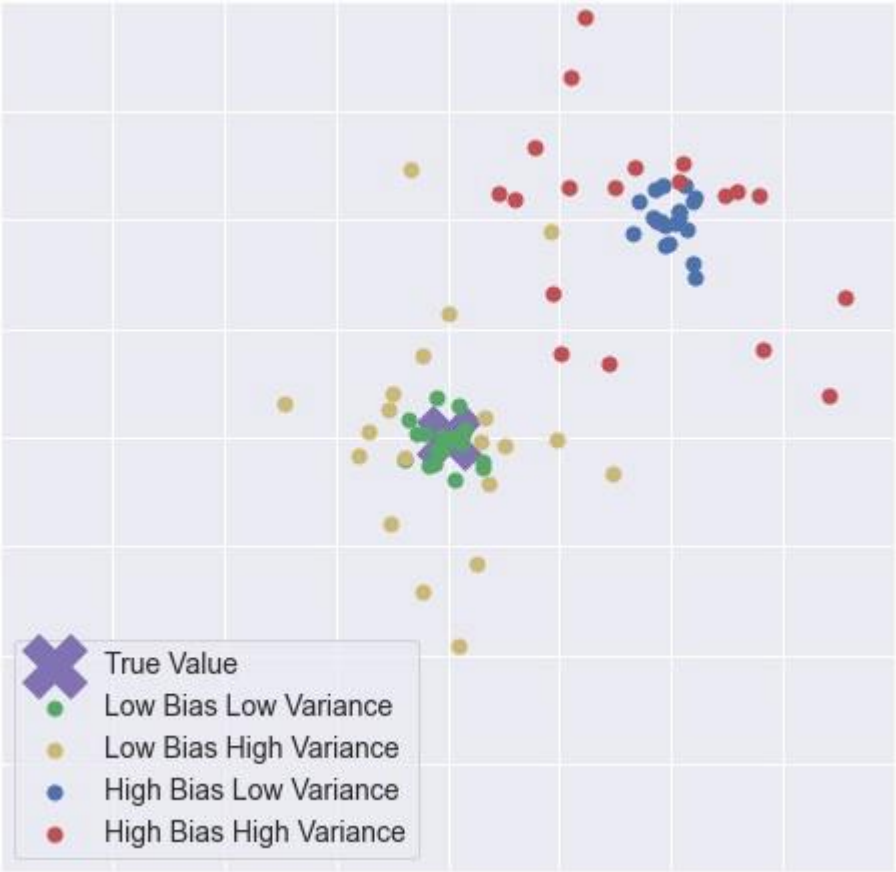
- low bias, low variance — is a good result, just right.
- low bias, **high variance** — **overfitting** — the algorithm outputs very different predictions for similar data.
- **high bias**, low variance — **underfitting** — the algorithm outputs similar predictions for similar data, but predictions are wrong (algorithm "miss").
- high bias, high variance — very bad algorithm. You will most likely never see this.



Bias and Variance options on four plots. Image by Author

All these cases can be placed on the same plot. It is a bit less clear than the previous one but more compact.

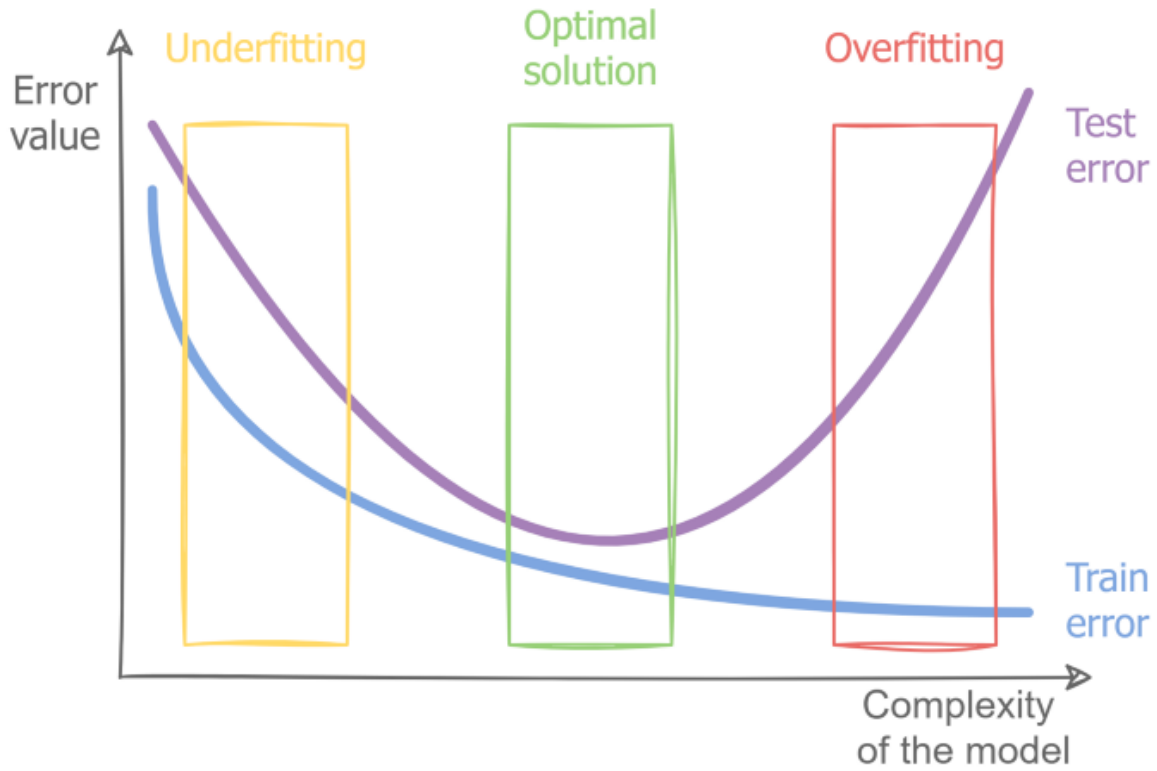
DEEPAK



Bias and Variance options on one plot. Image by Author

How to Detect Underfitting and Overfitting

Before we move on to the tools, let's understand how to "diagnose" underfitting and overfitting.



Train/test error and underfitting/overfitting. Image by Author

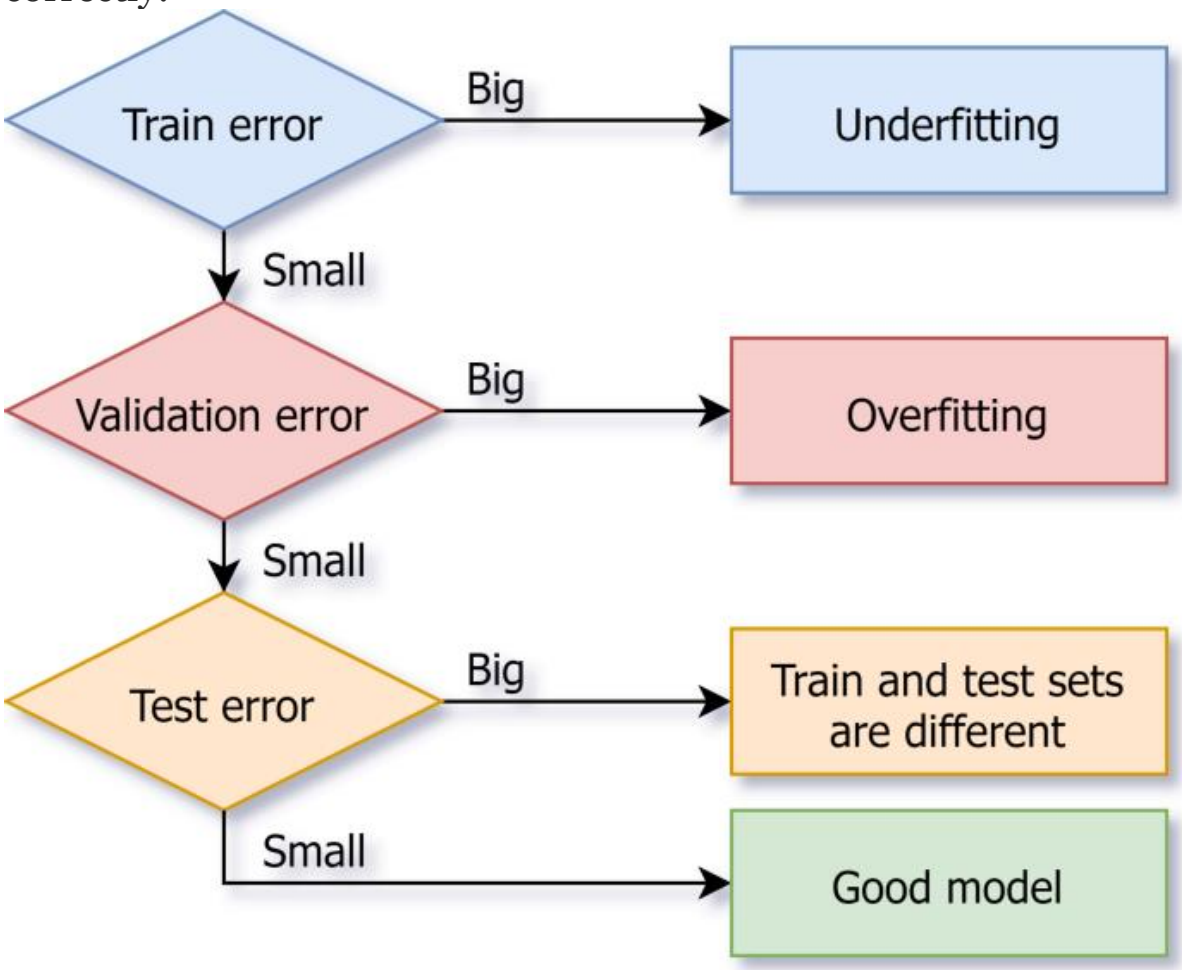
Underfitting means that your model makes accurate, but initially incorrect predictions. In this case, **train error is large** and **val/test error is large** too.

Overfitting means that your model makes not accurate predictions. In this case, **train error is very small** and **val/test error is large**.

When you find a **good model**, **train error is small** (but larger than in the case of overfitting), and **val/test error is small** too.

In the case above, the test error and validation error are approximately the same. This happens when everything is fine, and your *train, validation, and test data have the same distributions*. If

validation and test error are very different, then you need to get more data similar to test data and make sure that you split the data correctly.



How to detect underfitting and overfitting. Image by Author

General Intuition You Should Remember

As we remember:

- **underfitting** occurs when your model is **too simple** for your data.
- **overfitting** occurs when your model is **too complex** for your data.

Based on this, simple intuition you should keep in mind is:

- to fix **underfitting**, you should **complicate** the model.
- to fix **overfitting**, you should **simplify** the model.

More Simple / Complex Model

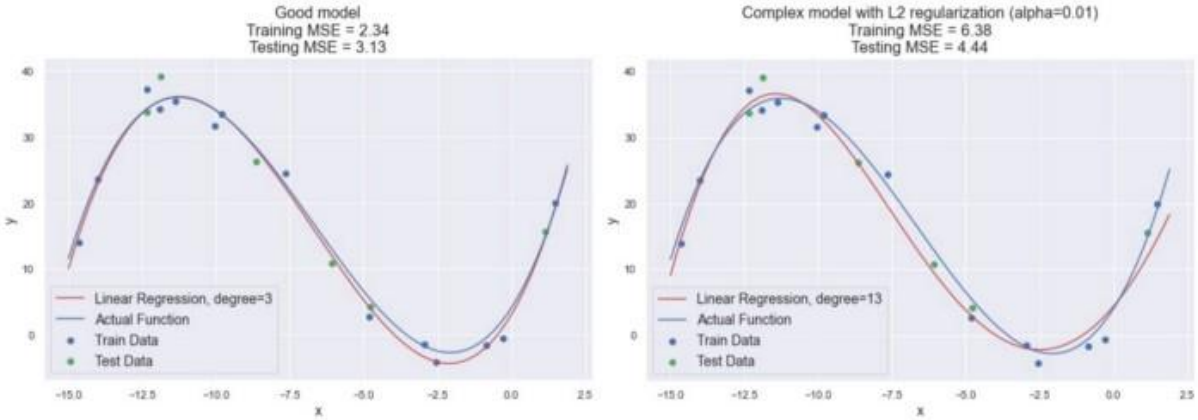
The easiest way that comes to mind based on the intuition above is to try a more simple or more complex algorithm (model).

To complicate the model, you need to add more parameters (*degrees of freedom*). Sometimes this means to directly try a more powerful model — one that is a priori capable to restore more complex dependencies (*SVM with different kernels instead of logistic regression*). **If the algorithm is already quite complex** (neural network or some ensemble model), **you need to add more parameters** to it, for example, increase the number of models in boosting. In the context of neural networks, this means adding more layers / more neurons in each layer / more connections between layers / more filters for CNN, and so on.

To simplify the model, you need contrariwise to reduce the number of parameters. Either completely change the algorithm (*try random forest instead of deep neural network*), or reduce the number of degrees of freedom. Fewer layers, fewer neurons, and so on.

More Regularization / Less Regularization

This point is very closely related to the previous one. In fact, **regularization is an indirect and forced simplification of the model**. The regularization term requires the model to keep parameters values as small as possible, so requires the model to be as simple as possible. Complex models with strong regularization often perform better than initially simple models, so this is a very powerful tool.



Good model and complex model with regularization. Image by Author

More regularization (simplifying the model) means increasing the impact of the *regularization term*. This process is strictly individual — depending on the algorithm, the regularization parameters are different (for example, **to reduce the regularization, the alpha for Ridge regression should be decreased, and C for SVM — increased**). So you should study the parameters of the algorithm and pay attention to whether they should be increased or decreased in a particular situation. There are a lot of such parameters — L1/L2 coefficients for linear regression, C and gamma for SVM, maximum tree depth for decision trees, and so on. In the context of neural networks, the main regularization methods are:

- Early stopping,
- Dropout,
- L1 and L2 Regularization.

You can read about them in this article.

Opposite, in the case when the model needs to be complicated, you should reduce the influence of regularization terms or abandon the regularization at all and see what happens.

More Features / Fewer Features

This may not be so obvious, but adding new features also complicates the model. Think about it in the context of a *polynomial regression* — adding quadratic features to a dataset allows a linear model to recover quadratic data.

Adding new "natural" features (if you can call it that) — obtaining new features for existing data is used infrequently, mainly due to the fact that it is very expensive and long. But you can keep in mind that sometimes this can help.

But artificial obtaining of additional features from existing ones (the so-called *feature engineering*) is used quite often for classical machine learning models. There are as many examples of such transformations as you can imagine, but here are the main ones:

- polynomial features — from x_1, x_2 to x_1, x_2, x_1x_2, x_1^2, x_2^2, … (`sklearn.preprocessing.PolynomialFeatures` class)

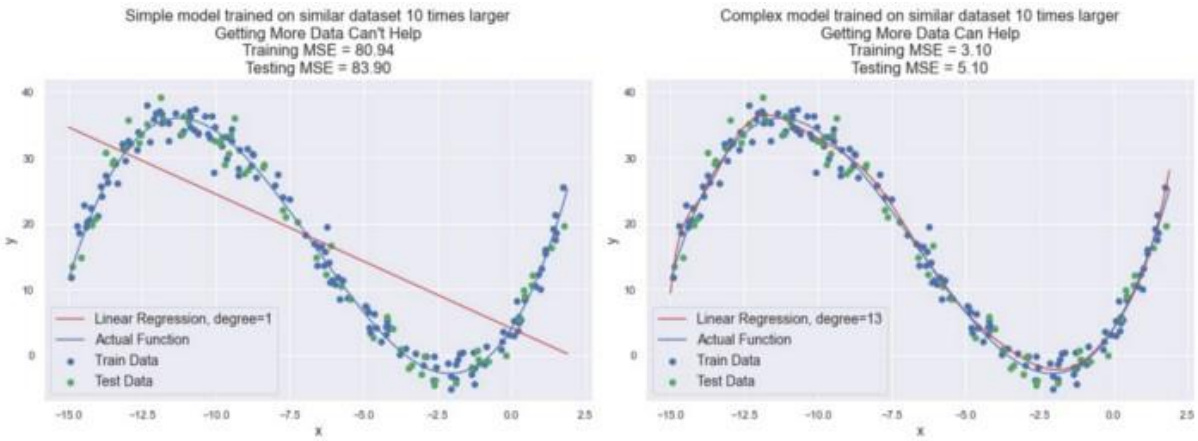
- log(x), for data with not-normal distribution
- ln(|x| + 1) for data with heavy right tail
- transformation of categorical features
- other non-linear data transformation (from length and width to area (length*width)) and so on.

If you need to simplify the model, then you should use a smaller quantity of features. First of all, remove all the additional features that you added earlier if you did so. But it may turn out that in the original dataset there are features that do not carry useful information, and sometimes cause problems. Linear models often work worse if some features are dependent — *highly correlated*. In this case, you need to use *feature selection* approaches to select only those features that carry the maximum amount of useful information.

It is worthwhile to say that in the context of neural networks, *feature engineering* and *feature selection* make almost no sense because **the network finds dependencies in the data itself**. This is actually why deep neural networks can restore such complex dependencies.

Why Getting More Data Sometimes Can't Help

One of the techniques to combat overfitting is to get more data. However, surprisingly, this may not always help. Let's generate a similar dataset 10 times larger and train the same models on it.



Why getting more data sometimes can't help. Image by Author

A very simple model (degree 1) has remained simple, almost nothing has changed. So **getting more data will not help in case of underfitting**.

Summary

Let's summarize everything in one table.

Techniques to fight underfitting and overfitting	
Underfitting	**Overfitting**
More complex model	More simple model
Less regularization	More regularization
Larger quantity of features	Smaller quantity of features
More data can't help	More data can help

Techniques to fight underfitting and overfitting. Image by Author

Well, better in two.

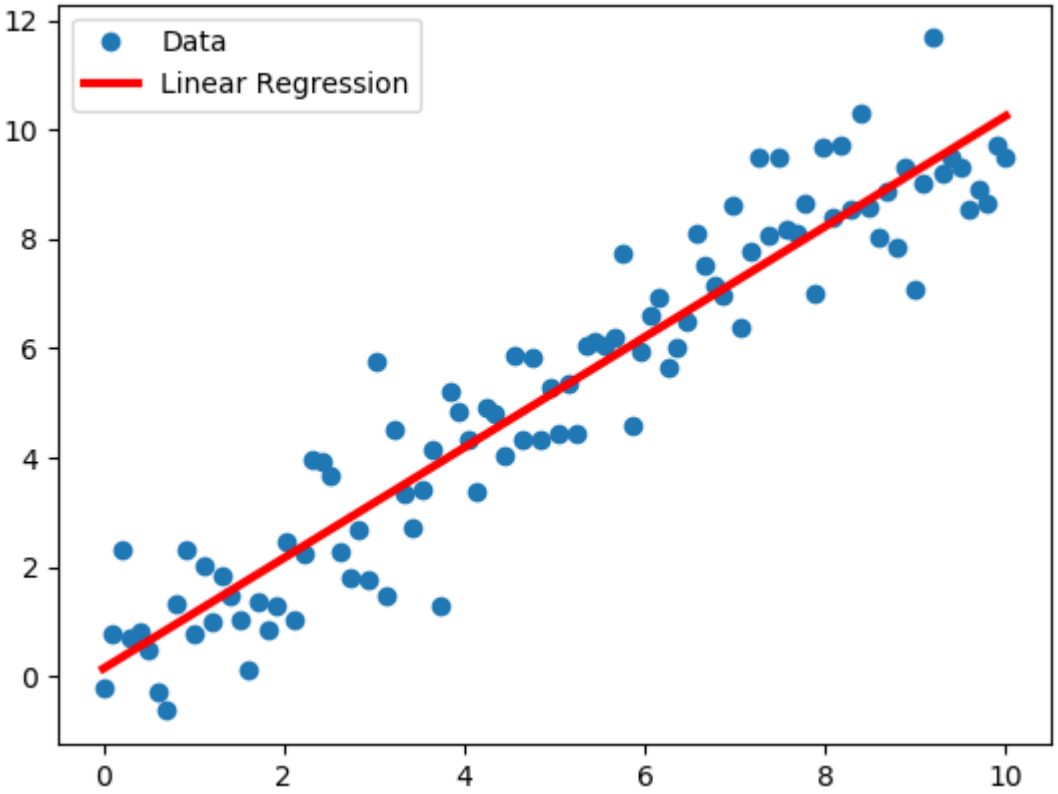
Techniques to fight underfitting and overfitting		
	Underfitting	**Overfitting**
Complexity of the model	More complex model Try a more powerful model with a larger number of parameters Ensemble learning More layers / number of neurons per layer	More simple model Try a less powerful model with a fewer number of parameters Less layers / number of neurons per layer
Regularization	Less regularization Decrease regularization	More regularization Increase regularization impact Early stopping, L1 / L2 regularization, dropout
Quantity of features	A larger quantity of features Get additional features, feature engineering, polynomial features, etc.	A smaller quantity of features Remove all additional features, feature selection
Data	Data cleaning, hold-out validation or cross validation. ~~Getting more data most likely will not help~~	Data cleaning, hold-out validation or cross validation. Getting more data most likely will help (data augmentation)

Techniques to fight underfitting and overfitting (extended). Image by Author

Some tools and techniques have not been covered in this article. For example, I consider **data cleaning** and **cross-validation** or **hold-out validation** to be common practices in any machine learning project, but they can also be considered as tools to combat overfitting.

You may notice that to eliminate underfitting or overfitting, you need to apply **diametrically opposite actions**. So if you initially "misdiagnosed" your model, you can spend a lot of time and money on empty work (for example, getting new data when in fact you need to complicate the model). That's why it is so important — hours of analysis can save your days and weeks of work.

What is Linear Regression ?



Simple Linear Regression is basically a modelling of linear relationship between linearly dependent variables that can be later used to predict dependent variable values for new independent variables

For this, we use the equation of the line : `y = m * x + c.`

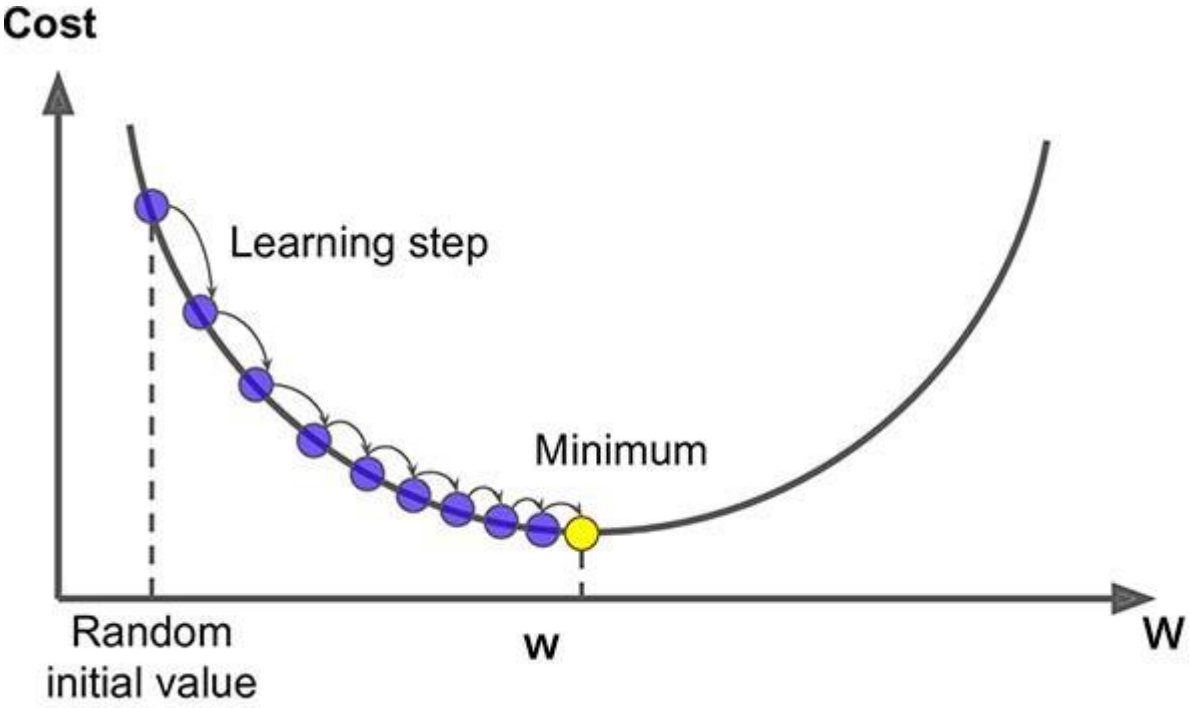
where y is the dependent variable and x is the independent variable

For example: We could predict the salary of a person with the years of experience of the person.

Here, Salary is the dependent variable and experience is the independent variable since, we are predicting salary with the help of experience.

When to use Linear Regression: Linear regression can be performed on data where there is a good linear relationship between dependent and independent variables.
The degree of linear relationship can be found with help of correlation.

Gradient Descent



Gradient Descent is a an optimization algorithm that can be used to find the global or local minima of a differentiable function.

Various Assumptions and definitions before we begin.

Let's say we have an independent variable x and a dependent variable y.
in order to form the relationship between these 2 variables, we have the equation:

y = x * w + b

where **w** is weight (or slope) ,
b is the bias (or intercept),
x is the independent variable column vector(examples),
y is the dependent variable column vector(examples)
Our main goal is to find the **w** and **b** that defines the relationship between variable **x** and **y** correctly. We accomplish this with the help of something called as the **Loss Function.**

Loss Function: A loss function is a function that signifies how much our predicted values is deviated from the actual values of the dependent variable.

Important Note: *we are trying to find the values for **w** and **b** such that it minimizes our loss function.*

Steps Involved in Linear Regression with Gradient Descent Implementation

1. Initialize the weight and bias randomly or with 0(both will work).
2. Make predictions with this initial weight and bias.
3. Compare these predicted values with the actual values and define the loss function using both these predicted and actual values.
4. With the help of differentiation, calculate how loss function changes with respect to weight and bias term.

5. Update the weight and bias term so as to minimize the loss function.

Implementation with Math

1	1.1	39343.00
2	1.3	46205.00
3	1.5	37731.00
4	2.0	43525.00
5	2.2	39891.00
6	2.9	56642.00
7	3.0	60150.00
8	3.2	54445.00
9	3.2	64445.00
10	3.7	57189.00

Example of Independent and dependent variables respectively

1 . Assumption

Let us say we have an **x** and **y** vectors like shown in the above pic (*The above one is only for an example*).

2. Initialize w and b to 0

$\mathbf{w} = 0, \mathbf{b} = 0$

3. Make some predictions with the current w and b. Of course, it's going to be wrong.

y_pred = x*w + b, where y_pred stands for predicted y values.
This y_pred will also be a vector like y.

4 . Define a loss function

loss = (y_pred — y)^2/n
where n is the number of examples in the dataset.
It is obvious that this loss function represents the deviation of the predicted values from the actual.
This loss function will also be a vector. But, we will be summing all the elements in that vector to convert it into scalar.

5. Calculate (∂(loss)/ ∂w)

The derivative of a function of a real variable measures the sensitivity to change of the function value with respect to a change in its argument.

We can use calculus to find how loss changes with respect to ***w***.

```
loss = (y_pred — y)²/n
loss = (y_pred² + y² — 2y*y_pred)/n (expanding the whole square)
      =>( (x*w+b)² + y² — 2y*(x*w+b))/n (substitute y_pred)
      => ((x*w+b)²/n ) + (y²/n) + ((-2y(x*w+b))/n) (splitting
the terms)
Let A = ((x*w+b)²/n )
Let B = (y²/n),
Let C = ((-2y(x*w+b))/n)A = ( x²w² + b² + 2xwb )/n (expanding)
∂A/∂w = ( 2x²w + 2xb )/n (differentiating)
∂B/∂w = 0 (differentiating)
C = (-2yxw — 2yb)/n
∂C/∂w = (-2yx)/n (differentiating)
```

So, ∂loss/∂w will be the addition of all these terms:

```
∂loss/∂w = (2x²w + wxb − 2yx)/n
=> (2x(x*w + b − y))/n
```

So, The derivative of loss with respect to ***w*** was found to be: **(2/n)*(y_pred — y)*x.** Let us call this dw.

If we perform the same differentiation for loss with respect to ***b***, we'll get:
(2/n)*(y_pred — y). Let us call this db.

This dw and db are what we call *"gradients"*

6. Update w and b

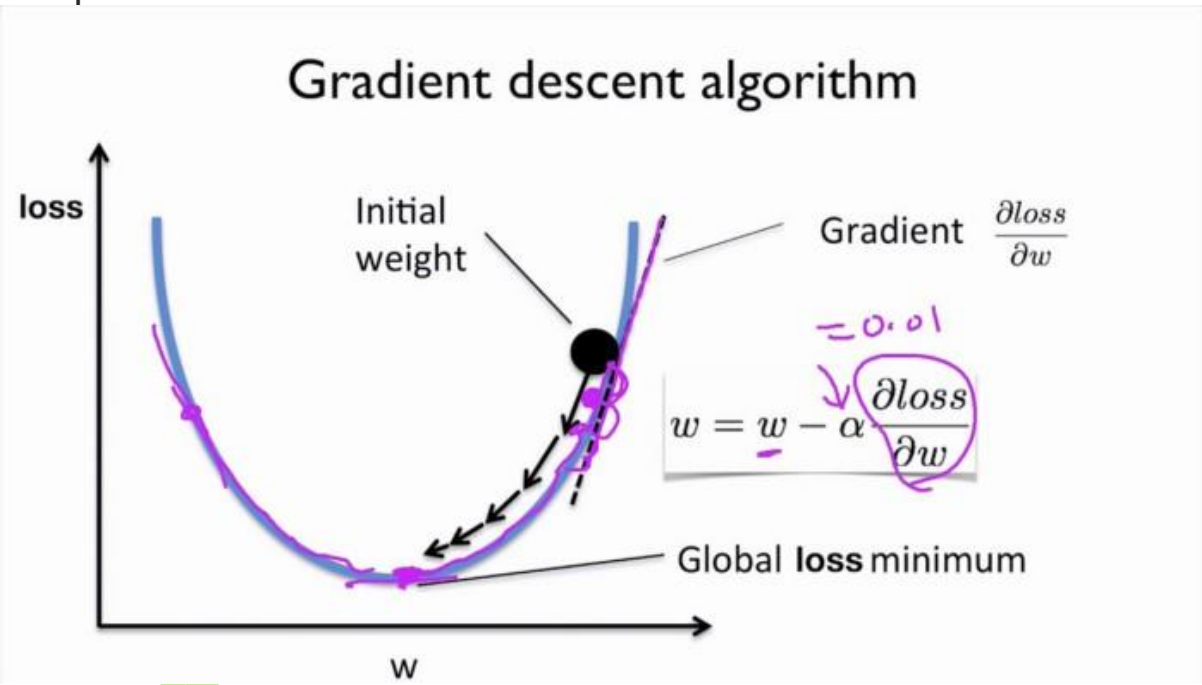


Figure 6.1

As we can see in the figure 6.1, if we initialize the weight randomly, it might not result in a global minimum of the loss function.
It is our duty to update the weights to the point where the loss is minimum. We have calculated **dw** above.

dw is nothing but the slope of the tangent of the loss function at point w.
Considering the initial position of **w**.

Important point to understand :

In the above diagram, The slope of the tangent of the loss will be positive as initial value of w is greater and it needs to be reduced so as to attain global minimum.
If the value of w is low and we want to increase it to attain global minimum, the slope of the tangent of loss at point w will be negative

We want the value of w to be a little lower so as to attain global minima of the loss function as shown in the figure 6.1.

We know that dw is positive in the above graph and we need to reduce w.
This can be done by:

```
w = w – alpha*dw
b = b – alpha*bw
```

where alpha is a small number ranging between 0.1 to 0.0000001 (approx) and this alpha is knows as the learning rate.
*Doing this, we can reduce **w** if slope of tangent of loss at **w** is positive and increase **w** if slope is negative*

7. Learning Rate

Learning rate alpha is something that we have to manually choose and it is something which we don't know beforehand. Choosing it is

a matter of trial and error.
The reason we do not directly subtract **dw** from **w** is because, it might result in too much change in the value of w and might not end up in global minimum but, even further away from it.

8. Training Loop

The process of calculating the gradients and updating the weight and bias is repeated for several times, which results in the optimized value for weight and bias.

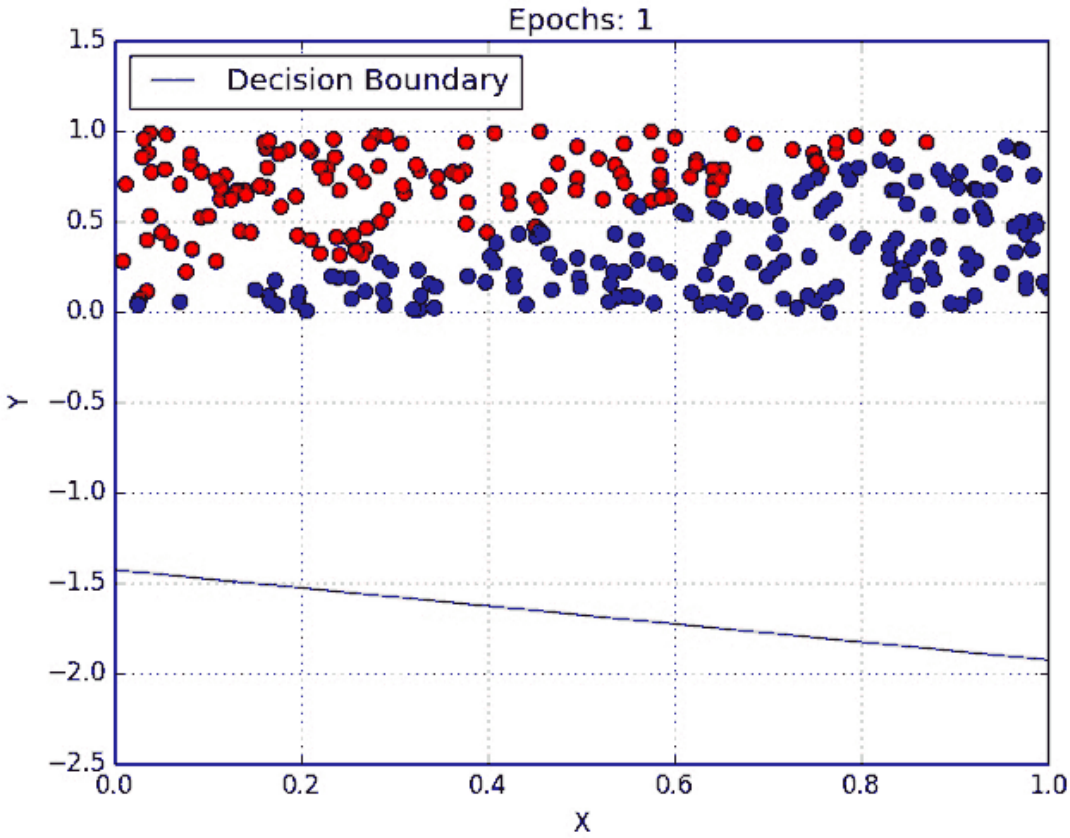
9. Prediction

After the training loop, the value of weight and bias is now optimized, which can be used to predict new values given new x values.

```
y = x*w + b
```

DEEPA

Introduction to Logistic Regression



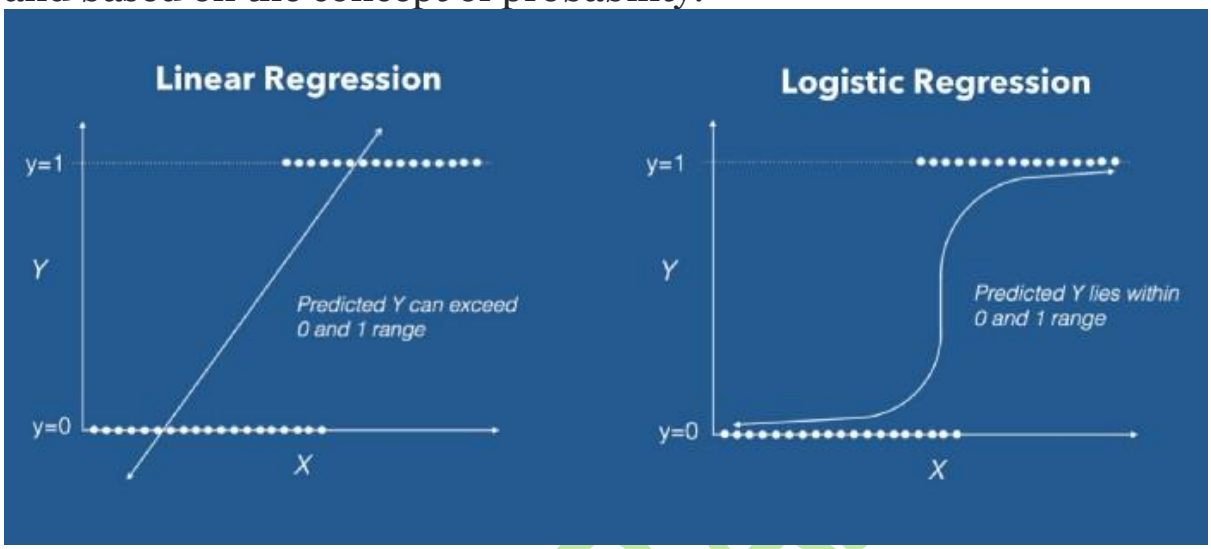
Logistic regression is a classification algorithm used to assign observations to a discrete set of classes. Some of the examples of classification problems are Email spam or not spam, Online transactions Fraud or not Fraud, Tumor Malignant or Benign. Logistic regression transforms its output using the logistic sigmoid function to return a probability value.

What are the types of logistic regression

1. Binary (eg. Tumor Malignant or Benign)
2. Multi-linear functions failsClass (eg. Cats, dogs or Sheep's)

Logistic Regression

Logistic Regression is a Machine Learning algorithm which is used for the classification problems, it is a predictive analysis algorithm and based on the concept of probability.



Linear Regression VS Logistic Regression Graph| Image: Data Camp

We can call a Logistic Regression a Linear Regression model but the Logistic Regression uses a more complex cost function, this cost function can be defined as the '**Sigmoid function**' or also known as the 'logistic function' instead of a linear function.

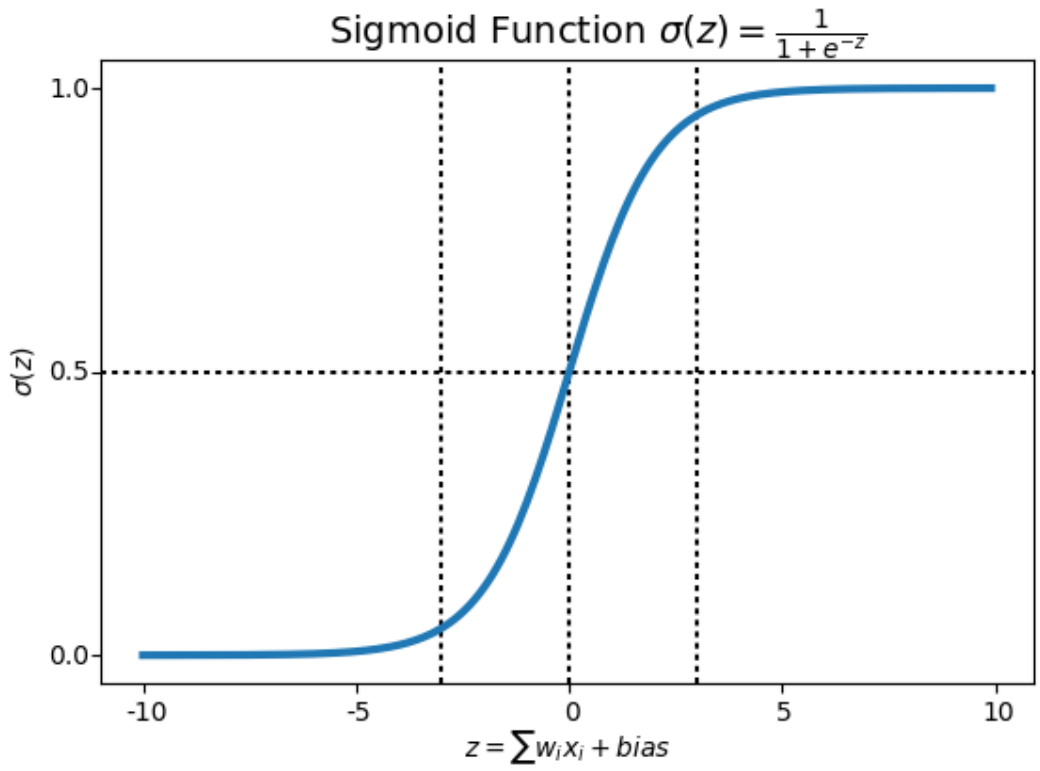
The hypothesis of logistic regression tends it to limit the cost function between 0 and 1. Therefore linear functions fail to represent it as it can have a value greater than 1 or less than 0 which is not possible as per the hypothesis of logistic regression.

$$0 \leq h_\theta(x) \leq 1$$

Logistic regression hypothesis expectation

What is the Sigmoid Function?

In order to map predicted values to probabilities, we use the Sigmoid function. The function maps any real value into another value between 0 and 1. In machine learning, we use sigmoid to map predictions to probabilities.



Sigmoid Function Graph

$$f(x) = \frac{1}{1+e^{-(x)}}$$

Formula of a sigmoid function | Image: Analytics India Magazine

Hypothesis Representation

When using *linear regression* we used a formula of the hypothesis i.e.

$h\Theta(x) = \beta_0 + \beta_1 X$

For logistic regression we are going to modify it a little bit i.e.

$\sigma(Z) = \sigma(\beta_0 + \beta_1 X)$

We have expected that our hypothesis will give values between 0 and 1.

$Z = \beta_0 + \beta_1 X$

$h\Theta(x) = \text{sigmoid}(Z)$

i.e. $h\Theta(x) = 1/(1 + e^{-(\beta_0 + \beta_1 X)})$

$$h\theta(X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

The Hypothesis of logistic regression